INDUSTRIAL ASSIGNMENT- 9 Running a full stack project
STUDENT INTERNSHIP AND PLACEMENT MANAGEMENT PORTAL

1. Introduction

A full stack project has three parts that must run together: the database, the backend and the frontend. In the Student Internship and Placement Management Portal the database is MySQL (Industrial Assignments 5 and 6), the screens follow the Figma design (Industrial Assignments 7 and 8), the backend is a Spring Boot application (Industrial Assignments 10 and 11) and the APIs are tested in Postman (Industrial Assignment 12). This assignment explains how to set up the tools, start the three parts in the correct order, connect them and check that they work together. It is the base for the coding assignments that follow.

2. Objectives

- Install and verify the tools required for the project.
- Create the MySQL database and load the 16 tables of Industrial Assignment 5.
- Create and run a Spring Boot backend that connects to the database.
- Create and run a React frontend that calls the backend.
- Run all three layers together and verify the connection.
- Prepare a folder structure and Git repository for the team.

3. Run-time Architecture

The browser shows the React application. The React application sends HTTP requests with JSON data to the Spring Boot backend. The backend uses Spring Data JPA to read and write the MySQL database. Postman sends requests directly to the backend to test the APIs without the frontend.

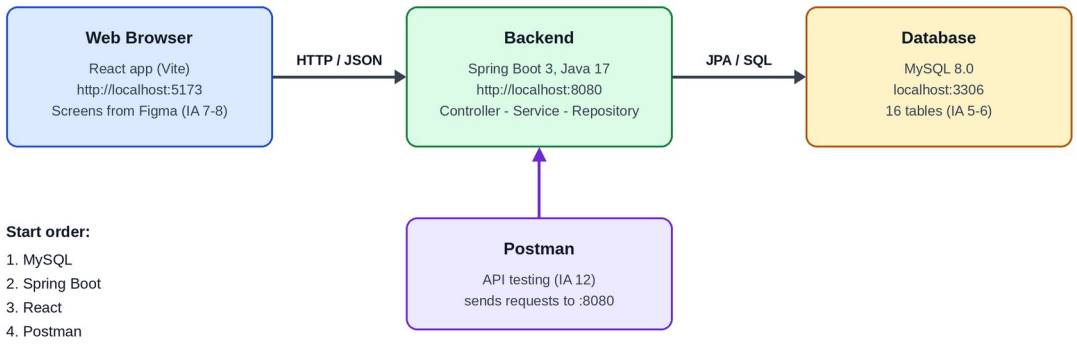


Figure 1: Parts of the project and their ports

Part	Technology	Port	Start command
Database	MySQL 8.0	3306	Start the MySQL service
Backend	Spring Boot, Java 17, Maven	8080	mvn spring-boot:run

Part	Technology	Port	Start command
Frontend	React with Vite, Node.js	5173	npm run dev
API testing	Postman	-	Open the collection

4. Software Requirements

Software	Version	Purpose and check command
JDK	17	Runs the backend. Check: java -version
Maven	3.8 or later	Builds the backend. Check: mvn -version
Node.js and npm	LTS version	Runs the frontend. Check: node -v and npm -v
MySQL Server and Workbench	8.0	Database. Check: mysql --version
Git and GitHub account	Latest	Version control. Check: git --version
VS Code or IntelliJ IDEA	Latest	Code editor
Postman	Latest	API testing
Figma	Web or desktop	Reference for the screens

5. Project Folder Structure

```
placement-portal/
  database/     schema.sql, sample_data.sql
  backend/      Spring Boot project (pom.xml, src/main/java, src/main/resources)
  frontend/     React project (package.json, src)
  postman/      collection and environment files
  docs/         Industrial Assignments 1 to 12
  README.md
```

6. Step 1: Set up the Database

1. Start the MySQL service and open MySQL Workbench or the command line.
2. Create the database and a user for the application.
3. Run the CREATE TABLE statements of Industrial Assignment 5 (save them as database/schema.sql).
4. Insert a few sample rows so that login and search can be tested.
5. Check that the 16 tables exist.

```
CREATE DATABASE placement_portal CHARACTER SET utf8mb4;
CREATE USER 'portal_user'@'localhost' IDENTIFIED BY 'ChangeThisPassword';
```

```
GRANT ALL PRIVILEGES ON placement_portal.* TO 'portal_user'@'localhost';
USE placement_portal;
SOURCE database/schema.sql;
SHOW TABLES;
```

Expected result: SHOW TABLES lists users, login_sessions, user_access_history, student_profiles, student_resumes, student_edit_history, companies, opportunities, opportunity_edit_history, opportunity_deletion_history, applications, application_status_history, interviews, offers, dashboard_preferences and report_history. The database name placement_portal is used in this document; use the name from your own scripts if it differs.

7. Step 2: Create and Run the Backend

Generate the project from Spring Initializr (start.spring.io) with Maven, Java 17 and the dependencies below, then open it in the editor.

Dependency	Reason
Spring Web	REST controllers
Spring Data JPA	Repositories and entities
MySQL Driver	Connection to MySQL
Validation	Input validation (business rule BR-02)
Spring Security Crypto (or Spring Security)	BCrypt password hashing (FR-03)

Configure the connection in src/main/resources/application.properties:

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/placement_portal?
serverTimezone=Asia/Kolkata
spring.datasource.username=portal_user
spring.datasource.password=ChangeThisPassword
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
```

The value validate makes Hibernate check the entities against the tables created in Step 1 instead of changing them. Allow the React application to call the backend by adding a CORS setting for http://localhost:5173, and add a simple test endpoint:

```
@RestController
public class HealthController {
  @GetMapping("/api/health")
  public Map<String, String> health() {
    return Map.of("status", "UP");
```

```
  }
}
```

Run the backend from the backend folder with mvn spring-boot:run. When the console shows "Started ... Application" and no error, open http://localhost:8080/api/health in the browser. The answer {"status":"UP"} shows that the backend runs. A database error in the console means the URL, user name or password is wrong.

8. Step 3: Create and Run the Frontend

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

Vite prints the address http://localhost:5173. Create a file src/api.js that holds the backend address, and call the health endpoint from a test component:

```
export const API = "http://localhost:8080";

fetch(API + "/api/health")
  .then(r => r.json())
  .then(d => console.log(d.status));
```

If the browser console shows "UP", the frontend, backend and database are connected. A CORS error means the allowed origin in the backend is wrong.

9. Step 4: Run All Parts Together

1. Start MySQL and confirm that the 16 tables exist.
2. Start the backend (mvn spring-boot:run) and open /api/health.
3. Start the frontend (npm run dev) and open http://localhost:5173.
4. Open Postman and send GET http://localhost:8080/api/health.
5. Build the first real feature end to end: registration (Task T01) from the S02 screen to the users table.

To stop the project, press Ctrl + C in the backend and frontend terminals and stop the MySQL service if it is no longer needed.

10. Verification Checklist

Check	Expected result	Done
java -version, mvn -version, node -v	Versions installed	
SHOW TABLES in placement_portal	16 tables listed	
Backend start	No error; port 8080 in use	

Check	Expected result	Done
GET /api/health in browser	{"status":"UP"}	
Frontend start	Page opens on port 5173	
Frontend calls backend	No CORS error in console	
Postman health request	Status 200 OK	
Git commit and push	Code visible on GitHub	

11. Common Problems and Solutions

Problem	Likely cause	Solution
Access denied for user	Wrong user name or password	Check the properties file and the GRANT statement
Communications link failure	MySQL is not running or port differs	Start the service; check port 3306
Schema-validation: missing table or column	Entity differs from the table	Compare entity fields with Industrial Assignment 6
Port 8080 already in use	Another program uses the port	Stop it or set server.port=8081
CORS error in the browser	Origin not allowed	Allow http://localhost:5173 in the backend
npm command not found	Node.js not installed	Install Node.js LTS and reopen the terminal
Unsupported class version	Wrong Java version	Set JAVA_HOME to JDK 17

12. Version Control

- Create one GitHub repository for the whole project and add a .gitignore for target/, node_modules/ and the properties file with passwords.
- Use the branches main and one feature branch for each task, for example feature/T01-register.
- Commit with clear messages such as "T01: add register API".
- Write in README.md the steps of Sections 6 to 9 so that any team member can run the project.

13. Evidence to Attach

Add your own screenshots to the submission: (1) SHOW TABLES output, (2) backend console after start, (3) /api/health in the browser, (4) frontend running in the browser, (5) Postman health request, and (6) the GitHub repository page.

14. Conclusion

The database, backend and frontend of the portal can now be started in order and verified with a health check. With this environment ready, the backend entities, repositories, services and controllers are developed in Industrial Assignments 10 and 11 and tested in Industrial Assignment 12.